Oxford Cambridge and RSA – style practice paper

GCSE (9–1) Computer Science

J277/02 Computational thinking, algorithms and programming

Topic 2.5: Programming languages and IDEs

Candidate forename		**Candidate surname**	
Centre number		**Candidate number**	

Practice questions in the style of OCR GCSE Computer Science (J277). Every question is original; the layout, command words, mark allocations and mark scheme follow the real exam format.

Time allowed	About 75 minutes (roughly 1 minute per mark)
Instructions	• Use black ink. HB pencil may be used for graphs and diagrams only. • Answer **all** the questions. • Write your answer to each question in the space provided. • Where the question asks you to write code or pseudocode, you may use any suitable high-level language or the OCR Reference Language; marks are for the logic, not exact syntax. • Do **not** use a calculator or any other electronic device unless told to.
Information	• The total mark for this paper is **69**. • The marks for each question are shown in brackets [ ]. • Quality of extended responses will be assessed in questions marked with an asterisk (*). • Where a question asks you to *state* or *give*, a short answer is enough; *describe* and *explain* need more detail.

Name: _____ Date: _____ Mark: _____ / 69

1 Programs can be written in high-level or low-level languages.

(a) State what is meant by a **high-level language**. **[2]**

(b) Give **one** example of a high-level language. **[1]**

(c) Give **two** examples of low-level languages. **[2]**

(d) Give **two** advantages of high-level languages compared with low-level languages. **[2]**

(e) Give **two** advantages of low-level languages compared with high-level languages. **[2]**

(f) Explain why a games console manufacturer might use assembly language to write part of the system software. **[2]**

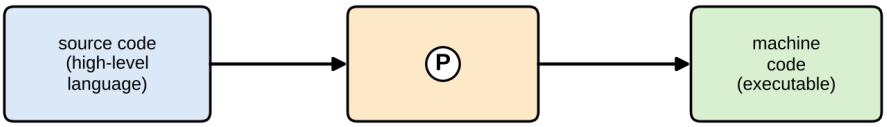
2 The CPU can only run instructions written in machine code.

(a) State what is meant by **machine code**. **[2]**

(b) Explain why programmers do not usually write programs in machine code. **[2]**

(c) A programmer writes a program in a high-level language. State what must happen before the CPU can run it. **[1]**

3 Programs written in high-level or assembly languages must be translated. The diagram shows the process.



(a) Name the three types of translator. **[3]**

(b) Component **P** translates the source code into machine code all at once and produces an executable file. Name **P**. **[1]**

(c) State which type of translator is used for assembly language. **[1]**

(d) Describe how an **interpreter** translates a program. **[2]**

4 A software company is choosing between a compiler and an interpreter for its programming language.

(a) Give **two** advantages of using a compiler. **[2]**

(b) Give **two** advantages of using an interpreter. **[2]**

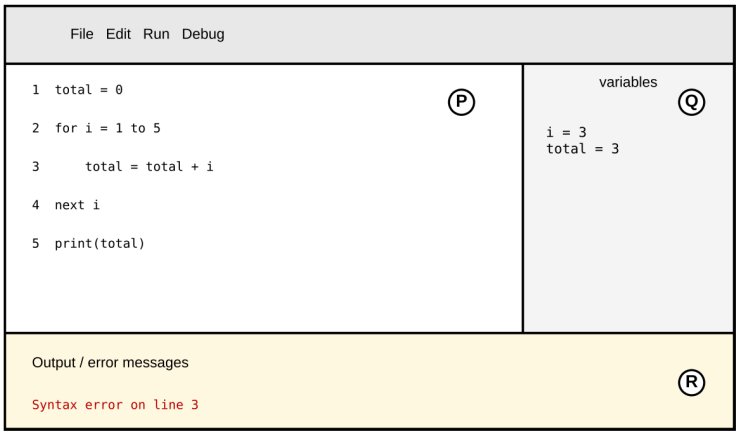
(c) Give **one** disadvantage of an interpreter. **[1]**

(d) A program contains an error on line 40 of 200. Explain how an interpreter and a compiler would each behave. **[3]**

(e) A programmer is developing a program and wants to test small changes quickly. Recommend a translator and justify your answer. **[2]**

(f) A company sells a program to customers and does not want them to see the code. Recommend a translator and justify your choice. **[2]**

5 Programmers use an integrated development environment (IDE). The diagram shows part of an IDE. Three areas are labelled **P**, **Q** and **R**.



(a) State what is meant by an **IDE**. **[2]**

(b) Name the IDE feature shown at **P**. **[1]**

(c) The area labelled **R** shows the message 'Syntax error on line 3'. Name this IDE feature. **[1]**

(d) The area labelled **Q** shows the current value of each variable as the program is run. Name the debugging tool that uses this. **[1]**

(e) Describe **two** features of a code editor that help a programmer. **[2]**

(f) Describe how a **breakpoint** and **stepping** can be used to find a logic error. **[3]**

6 A programmer is writing a computer program.

(a) State **two** tools in an IDE that help a programmer to find and correct errors. **[2]**

(b) State what is meant by a **run-time environment** in an IDE. **[1]**

(c) Explain how an IDE can save time for a programmer developing a large program. **[3]**

(d) A student says: 'An IDE translates the source code, so I do not need a compiler or interpreter.' Explain why the student is wrong. **[2]**

7 Extended response questions.

(a) *A company is developing a new app for smartphones. Discuss whether the developers should use a compiler or an interpreter, and what IDE features would help them. Include a justification in your answer. **[8]**

(b) *Compare high-level and low-level languages. In your answer you should consider the uses of each, and the way they are translated. **[8]**

END OF QUESTIONS

MARK SCHEME – Topic 2.5: Programming languages and IDEs

How to use this mark scheme: mark your own work in a different colour. A bullet (•) in the Answer column is a separate marking point worth one mark unless stated. A slash (/) separates alternative answers that are equally acceptable. Words in brackets are not needed. **Accept** = acceptable; **Ignore** = neither credited nor penalised; **Do not accept** = contradicts or spoils the answer. For programming questions, any valid solution that works is acceptable and syntax errors are not penalised if the intent is clear. The code in the answers uses the OCR Reference Language style.

Question	Answer	Marks	Guidance
1(a)	• a language that is close to human language / uses English-like commands ; • must be translated into machine code before the computer can run it ;	2 AO1 2.5.1	
1(b)	Python / Java / C# / Visual Basic / JavaScript / C++ ;	1 AO1 2.5.1	
1(c)	• machine code ; • assembly language ;	2 AO1 2.5.1	
1(d)	• easier to read, write and understand ; • easier / quicker to debug and maintain ; • portable: the same code can run on different types of processor ; • one line of code does the work of many low-level instructions ;	2 AO1 2.5.1	Any two.
1(e)	• can directly control / access hardware and memory ; • programs can be faster / use less memory ; • machine code needs no translation ;	2 AO1 2.5.1	Any two.
1(f)	• to directly control the hardware / efficient use of processor ; • so the software runs faster / uses less memory ;	2 AO3 2.5.1	
2(a)	• instructions written in binary (1s and 0s) ; • that the CPU can execute directly without translation ;	2 AO1 2.5.1	
2(b)	• very difficult to read / write / understand ; • easy to make errors / hard to debug / specific to one type of CPU ;	2 AO2 2.5.1	
2(c)	it must be translated into machine code ;	1 AO1 2.5.2	
3(a)	• compiler ; • interpreter ; • assembler ;	3 AO1 2.5.2	
3(b)	compiler ;	1 AO2 2.5.2	
3(c)	assembler ;	1 AO1 2.5.2	
3(d)	• translates one line / statement at a time ; • and runs it immediately, before moving to the next line (no executable produced) ;	2 AO1 2.5.2	

Question	Answer	Marks	Guidance
4(a)	• the compiled program runs faster / no translation at run time ; • the executable can be distributed without sharing the source code ; • translation done once, so the program can be run many times ; • all errors are reported after compilation (gives a list of errors) ;	2 AO1 2.5.2	Any two.
4(b)	• stops at the line with an error so easier to debug / test ; • program can be run immediately without waiting for compilation ; • the same source code runs on any computer that has the interpreter ;	2 AO1 2.5.2	Any two.
4(c)	• program runs slower because translation occurs every time ; • source code must be supplied to the user / interpreter must be installed ;	1 AO1 2.5.2	Any one.
4(d)	• interpreter runs lines 1 to 39 then stops at line 40 reporting the error ; • compiler translates the whole program and gives error report(s) ; • no executable file produced until the error is fixed ;	3 AO3 2.5.2	
4(e)	• interpreter ; • runs the code immediately without full compilation / easier to find errors line by line ;	2 AO3 2.5.2	
4(f)	• compiler ; • customers receive only the machine code executable, not source code / runs faster ;	2 AO3 2.5.2	
5(a)	• software used to write / develop programs ; • that provides several tools in one place (editor, translator, debugger) ;	2 AO1 2.5.3	
5(b)	code editor / source code editor ;	1 AO2 2.5.3	
5(c)	error diagnostics / error messages ;	1 AO2 2.5.3	
5(d)	variable watch / variable inspector (during step-through) ;	1 AO2 2.5.3	Accept: stepping / watch window.
5(e)	• syntax highlighting / colouring keywords ; • auto-completion / auto-indentation ; • line numbers / bracket matching ; • search and replace ;	2 AO1 2.5.3	Any two.
5(f)	• a breakpoint stops the program at a chosen line ; • stepping runs one line at a time ; • the programmer checks the values of variables after each line to see where they become wrong ;	3 AO2 2.5.3	
6(a)	• error diagnostics ; • breakpoints ; • stepping ; variable watch ; run-time environment / debugger ;	2 AO1 2.5.3	Any two.
6(b)	lets the programmer run / test the program inside the IDE without leaving it ;	1 AO1 2.5.3	

Question	Answer	Marks	Guidance
6(c)	• all the tools (editor, translator, debugger) in one place ; • features such as auto-complete / syntax highlighting reduce typing and mistakes ; • errors are found and fixed quickly using diagnostics and debugging tools ;	3 AO2 2.5.3	
6(d)	• an IDE contains / uses a translator (compiler or interpreter) ; • the translator does the translation, not the IDE itself ;	2 AO3 2.5.3	
7(a)	**Level 3 (high), 6–8 marks**: Thorough knowledge and understanding of the topic with detailed, accurate and relevant points. A balanced, well-structured answer reaching a justified conclusion where asked. Technical terms used correctly throughout. **Level 2 (mid), 3–5 marks**: Reasonable knowledge and understanding. Some relevant points made, but the answer lacks detail, balance or a clear structure in places. **Level 1 (low), 1–2 marks**: Basic knowledge. A few simple, mostly isolated points with little or no development or linking. **Level 0, 0 marks**: No response or no response worthy of credit. **Indicative content** (credit any other valid points): • compiler: translates the whole program at once into an executable; faster to run; source code protected; but compile step needed after each change • interpreter: translates and runs line by line; useful during development as errors found immediately and code can be tested quickly; slower to run; source code must be available • final released app usually compiled for speed / security; testing may use an interpreter • IDE features: code editor (syntax highlighting, auto-complete, indentation); error diagnostics • debugging tools: breakpoints, stepping, variable watch; run-time environment to test in the IDE • IDE improves productivity and reduces errors; one place for all tools • justified conclusion linking the choice of translator to the stage of development	8 AO1 / AO2 / AO3 2.5.2 / 2.5.3	

Question	Answer	Marks	Guidance
7(b)	**Level 3 (high), 6–8 marks**: Thorough knowledge and understanding of the topic with detailed, accurate and relevant points. A balanced, well-structured answer reaching a justified conclusion where asked. Technical terms used correctly throughout. **Level 2 (mid), 3–5 marks**: Reasonable knowledge and understanding. Some relevant points made, but the answer lacks detail, balance or a clear structure in places. **Level 1 (low), 1–2 marks**: Basic knowledge. A few simple, mostly isolated points with little or no development or linking. **Level 0, 0 marks**: No response or no response worthy of credit. **Indicative content** (credit any other valid points): • high-level: English-like, easy to read and write, quicker to develop / debug, portable between processors • high-level must be translated by a compiler or interpreter into machine code; one statement becomes many machine code instructions • low-level: machine code (binary) and assembly (mnemonics); close to the hardware; specific to one processor architecture • assembly translated by an assembler; machine code needs no translation and can be executed directly by the CPU • low-level gives precise control of hardware and memory; programs can be fast and small; used for embedded systems, device drivers, operating system parts • low-level is hard to write, read, debug and maintain, so development takes longer and costs more • high-level used for most software (apps, websites, games); conclusion matching language to purpose	8 AO1 / AO2 / AO3 2.5.1 / 2.5.2	

Where marks are lost in Topic 2.5

Use this page after marking. Tick the ones that cost you marks, then redo those questions closed-book.

- ☐ **Compiler vs interpreter:** compiler = whole program → executable, *faster to run*; interpreter = line by line, *easier to debug*, slower to run.
- ☐ **Assembler** translates *assembly language*. Machine code needs no translation.
- ☐ **Always say WHY a feature matters:** 'breakpoint' alone earns nothing; 'stops the program at a chosen line so values can be checked' does.
- ☐ **High-level = portable, easy; low-level = fast, close to hardware, specific to one CPU.**
- ☐ **IDE features to know:** editor, error diagnostics, run-time environment, translator, debugging tools (breakpoints, stepping, variable watch).
- ☐ **Match translator to scenario:** testing in development → interpreter; selling a finished program → compiler.
- ☐ **Do not confuse** an IDE with a translator: an IDE *contains* tools; the translator does the translation.